



Pearson

**Pearson Level 3 Alternative Academic Qualification BTEC National in Computing
(Extended Certificate)**

Version 3

Time 36 hours (approximately)

**Paper
reference**

XXXX/XX

Computing

UNIT 4: Practical Programming

**Assignment Title: Software Development for a Given Brief
Pearson Set Assignment Brief**

You must have:

Data files:

Hotdog vendors: Hotdogs.txt (enclosed); HotdogsDataDescriptions.docx (enclosed)

A computer with:

a programming language translator capable of producing executable code

an integrated development environment, including an editor, which can highlight syntax errors, and a debugger, which provides breakpoints, stepping, and variable inspection

a linter for the programming language source code files

the internet for research and tutorials

productivity software, including word-processor, spreadsheet, presentation, diagramming, and PDF reader applications.

W80512A

©2025 Pearson Education Ltd.
1/1/1/1

Continue ►

Instructions for teachers

Working through the bank of Pearson Set Assignment Brief versions

- The bank of assignments for this unit comprises four Pearson Set Assignment Briefs (versions 1 – 4) – this document is version 3.
- Centres must ensure that the Pearson Set Assignment Brief versions are allocated at cohort level, with all students in a given academic year being allocated the same option.
- Centres must ensure that these versions are **changed each year**, moving through **all** of the options available before any are repeated.
- Centres are permitted to choose the order that the versions are administered.

Preparation

- Centres must ensure students have access to the following resources:
 - Hotdog Vendors: Hotdogs.txt (enclosed);
HotdogsDataDescriptions.docx (enclosed)
 - A computer with:
 - a programming language translator capable of producing executable code
 - an integrated development environment, including an editor, which can highlight syntax errors, and a debugger, which provides breakpoints, stepping, and variable inspection
 - a linter for the programming language source code files
 - the internet for research and tutorials
 - productivity software, including word-processor, spreadsheet, presentation, diagramming, and PDF reader applications.
- No adaptation of this Pearson Set Assignment Brief is permitted.

Conditions of assessment

- Following the preparatory and planning work, tasks may be completed without direct supervision, outside of the classroom.
- Centres must ensure that there is sufficient supervision of students to enable work to be authenticated. Students must complete an authentication sheet and submit this along with their work.
- The estimated hours stated for completion of this Pearson Set Assignment Brief is 36 hours, including supervised and unsupervised time. Whilst this duration is intended to be used for guidance only, it should be used as an indication of how long students should spend on the assignment.
- Teachers are not permitted to provide feedback to students about how to improve their work or guide them to solutions to any questions or problems in the tasks.

Outcomes for submission: Students are required to complete, and submit evidence for, all tasks. Information regarding evidence requirements is presented within each task.

Instructions for students

You must carefully read the Pearson Set Assignment Brief.

This Pearson Set assignment brief will consist of two tasks.

You must submit evidence for each of the tasks.

You must work independently and must not share your work with other students. All evidence must be your own. You must complete an authentication sheet and submit this along with your work.

Any sources of information, ideas, text, audio and/or visual assets created by others that you include in your work must be clearly identified and referenced including the source of the material. Using the work of others as your own or without proper acknowledgement is considered plagiarism and can result in disqualification from the assessment.

You may ask your teacher for support if you have questions about the requirements of tasks, what evidence you need to produce and any resources you are allowed to access such as internet to source audio/visual content. Teachers cannot give you feedback about how to improve your work or guide you to solutions to any questions or problems in the tasks.

You must ensure that the images/scans, audio or video files you use as evidence are clear and of good quality, and clearly show the quality/nature of the work being demonstrated.

You must ensure that digital files, if produced, are saved in an accessible format that does not require specialist software to access.

Introductory context

Computing professionals produce software solutions to real-life, complex problems using computational thinking skills.

In this assignment you will use these skills to develop software to meet the requirements of the given brief:

- Hotdog Vendors

Further details of the brief are given in Appendix 1.

You will be required to use computational thinking skills to effectively analyse a problem, decompose it into its component parts, design data structures, and algorithms. You will then implement the design, using a systematic approach to ensure functional outcomes that meet requirements. You will produce documentation detailing the development process and maintain a repository of program code versions.

You must complete both tasks in the PSAB in parallel. As you produce coding for Task 1, you will need to update your report for Task 2. You will need to work in this way throughout the assignment. You must submit evidence for each of the tasks.

Tasks

Task 1

Software development

You will produce a coded solution to meet the requirements set out in the **Requirements** section for your brief in Appendix 1.

The solution will use the brief and supplied data, as a starting point. You are free to invent user scenarios and make implementation decisions that help you meet the requirements. You may include **additional** functionality beyond the requirements if you wish.

You are advised to:

- Select and use appropriate data structures
- Select and use appropriate validation
- Update your report for Task 2 as you work through this task.

Checklist of Evidence Required

- A document describing the development of the software solution.
- A collection of saved program code representing milestones.

Assessment criteria for this task

Pass	Merit	Distinction
A.P1 Develop software to meet some requirements that incorporates some appropriate: – inputs and outputs – data structures – sorting and searching algorithms.	A.M1 Employ a range of good practices and effective systematic approaches to develop software that meets most requirements and incorporates mostly appropriate: – inputs and outputs – data structures – sorting and searching algorithms.	A.D1 Consistently employ a range of good practices effectively and use efficient systemic approaches to develop software that fully meets requirements and incorporates: – well-chosen inputs and outputs – a range of data structures – a range of sorting algorithms and searching algorithms.
A.P2 Employ some good practices in programming during software development.		
A.P3 Apply a systematic approach to software development.		

Synopticity

To complete this task, you are expected to draw from knowledge and understanding about Content area A: Introduction to programming, logic and number, learned in Unit 1: Programming Fundamentals.

Transferable skills

By meeting the Pass grade criteria in this task, you will have demonstrated the following transferable skill:

- Creativity and Innovation

Task 2

Development report

You will manage the development of a software solution and demonstrate adherence to good programming practices. You will document this process in a single word-processed document.

Requirements

- Identify features and break features down into user stories
- Develop a single feature/user story at a time
- Produce a demonstrable functional software solution for each feature
- Backup up each feature completion as a milestone
- Ensure your code and logic is clear by using comments, white space, etc.
- Ensure separation of concerns, by using subprograms and interfaces
- Ensure your code is robust, by using exception handling and validation, where appropriate
- Demonstrate the use of linting
- Develop unit test for two appropriate subprograms
- Provide a reference list of sources used to help you complete the project.

You are advised to update your report for this task as you work through Task 1.

Checklist of Evidence Required

- A single word-processed document describing the development of the software solution.

Assessment criteria for this task

Pass	Merit	Distinction
B.P4 Employ software development management strategies, document functional milestones, and maintain a repository of files containing computer program code.	B.M2 Employ effective software development management strategies, fully document functional milestones, and maintain a repository of files demonstrating functional milestones.	B.D2 Employ fully appropriate and effective software development management strategies, fully and effectively document functional milestones, and maintain a repository of files effectively demonstrating functional milestones.

Other resources needed to complete this assignment

- Data files:
 - Hotdog Vendors:
 - Hotdogs.txt (enclosed)
 - HotdogsDataDescriptions.docx (enclosed)

Appendices

Appendix 1

Brief: Hotdog Vendors

A wholesale distribution company provides hotdog vendors with products and supplies.

The wholesale distribution company wants to maintain data about the products it supplies to different vendors. Someone at the company thought that a computer program might be useful. The employees want to search for vendors and make changes to the data about the vendors. One employee asked for a sorted list of the vendors. Someone else suggested that it would be useful to know how efficient the program was, as the company is hoping to expand their market to include 4 000 vendors nationally. Like most clients, they do not really know what they want, but have asked you to make a prototype for them.

The information about the vendors and supplies is provided in the comma separated value text file, **Hotdogs.txt**. The columns hold the vendor identifier, the vendor name, the year and week identifier, the number of vegan hotdogs supplied, the number of meat hotdogs supplied, the weight of onions ordered (kg), and the volume of ketchup ordered (litres).

The word-processed document, **HotdogsDataDescriptions.docx**, describes how each field in Hotdogs.txt is formed. This can be used for validation rules.

Produce a coded solution to meet the requirements set out in the **Requirements** section. The solution will use the supplied data, as a starting point. You are free to make implementation decisions that help you meet the requirements. You may include **additional** functionality beyond the requirements if you wish.

Requirements

- Linear search the data when it is sorted
- Linear search the data when it is unsorted
- Search the data using a binary search
- Sort the data using a bubble sort
- Sort the data using a quick sort
- Compare the efficiency (execution time) of the different searches
- Compare the efficiency (execution time) of the different sorts
- Provide analysis for the user (most productive vendor, number of vegan hotdogs versus meat hotdogs supplied, vendor using the least amount of ketchup, etc.)
- Save results of analysis to an output file.